ADVANCED STATIC ANALYSIS

MALWARE ANALYSIS AND INCIDENT FORENSICS

M.Sc. in Cyber Security

MALWARE ANALYSIS

M.Sc. in Engineering in Computer Science

A.Y. 2026/2027



SAPIENZA
UNIVERSITÀ DI ROMA



CIS SAPIENZA
CYBER INTELLIGENCE AND INFORMATION SECURITY

ADVANCED STATIC ANALYSIS

From disassembly to malware behavior - *Reverse engineer the sample under analysis* following this workflow:

- ▶️ Use tools like IDA to disassemble and inspect the code

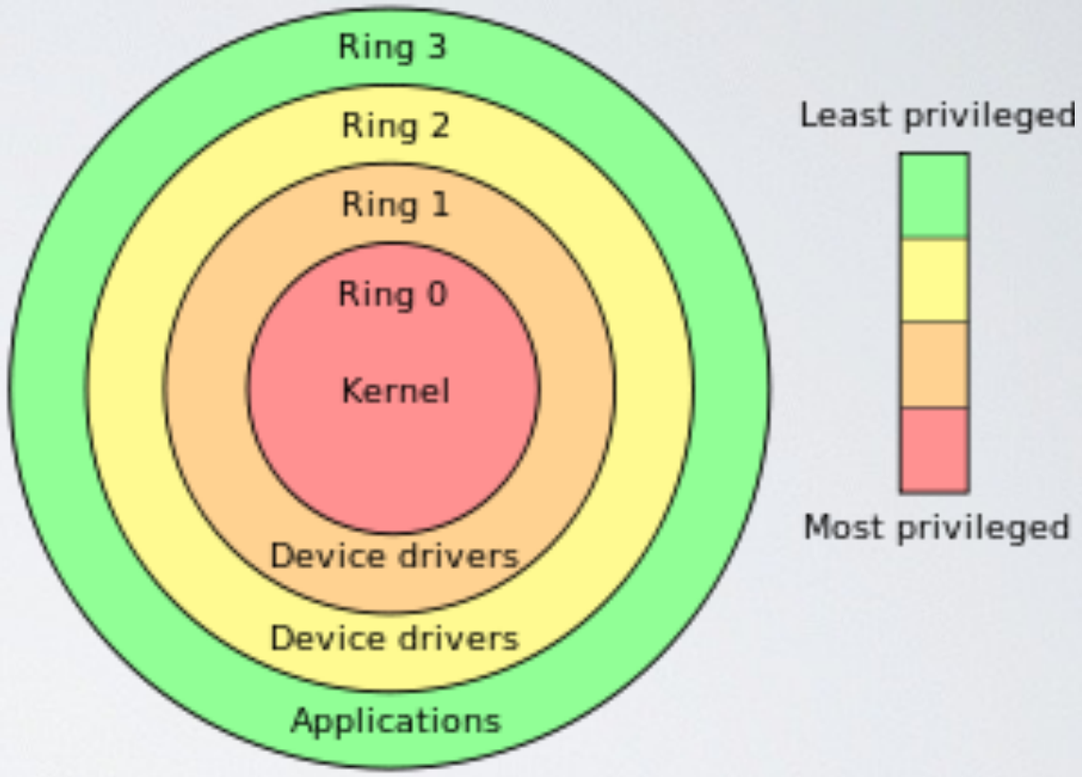
  Features to skim through code, search for specific snippets, interpret the code flow, etc.

- ↪️ Identify relevant API calls and functions
- ↪️ Inspect call sites, arguments, and control flow
- ↪️ Understand the relevant malware behavior
- ↪️ Extract IoCs

To simplify this part, we will assume that the malware code is not obfuscated or crippled in strange ways

PRIVILEGE BOUNDARY

- Ring 0: Kernel Mode
- Ring 3: User mode
- Rings 1 and 2 are not used by Windows



Nearly all code runs in user mode, except OS and hardware drivers

User-mode code can manipulate hardware only through the Windows API (i.e., no direct access to it). Access to some CPU instructions is restricted

USER MODE PROCESSES

- Each process has its own memory (isolated address space), security permissions, and resources
- If a user-mode program executes an invalid instruction and crashes, Windows can reclaim the resources and terminate the program

CALLING THE KERNEL

- It is not possible to jump directly from user mode to the kernel
- Special instructions (`sysenter`, `syscall`, `int 0x2e`) serve as gates that invoke predefined functions through lookup tables
  - Their presence is an indicator that the code runs functions at the kernel level

KERNEL PROCESSES

- All kernel processes share resources and memory addresses
- Fewer security checks, but PatchGuard as integrity mechanism
- If kernel code executes an invalid instruction, the OS crashes with the Blue Screen of Death (BSoD)
- Some security solutions are supported by kernel-mode components
- Kernel-mode malware is more rare, specialized, and sophisticated than user-mode malware. Rootkits are a common example

THE NATIVE API

- Lower-level interface for interacting with Windows
  - **`ntdll.dll`** manages interactions between user space and the kernel
  - ntdll functions make up the Native API
- Rarely used by goodware
- Popular among malware writers as it can be more powerful and stealthier than Windows API calls
- Limited official documentation

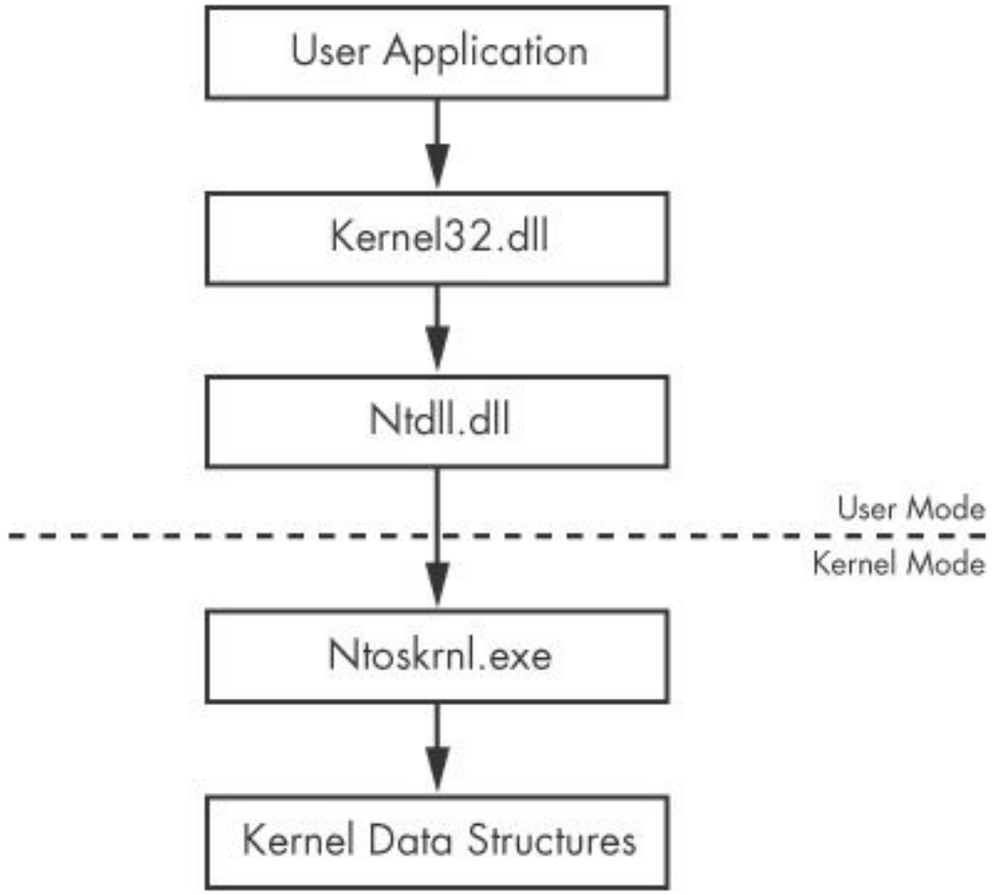


Figure 8-3. User mode and kernel mode

POPULAR NATIVE API CALLS IN MALWARE

- Examples of Native API calls that can be used to get information about the system, processes, threads, handles, and other items:
  - `NTtQuerySystemInformation`
  - `NTtQueryInformationProcess`
  - `NTtQueryInformationThread`
  - `NTtQueryInformationFile`
  - `NTtQueryInformationKey`
- Provide more information than any available Win32 APIs from DLLs

WINDOWS API (WIN32)

The **Win32 API** provides applications with functions to interact with Windows and access operating system services and resources.

Programs normally interact with OS resources through API functions exported by system DLLs.

Concepts we will cover next:

- Types and Hungarian notation
- Handles
- File-system functions
- Special files

TYPES AND HUNGARIAN NOTATION

Windows API has its own names to represent C data types

- DWORD for 32-bit unsigned integers, WORD for 16-bit unsigned integers, etc.

Hungarian Notation

- Variable names are prefixed to indicate their type or intended use. For example, variables that contain a 32-bit unsigned integer start with the prefix dw:

Type (prefix)	
WORD (w)	16-bit unsigned value
DWORD (dw)	32-bit unsigned value
Handle (H)	A reference to an object
Long Pointer (LP)	Points to another type

HANDLES

Items opened or created in the OS, like

- Process, menu, file, window...

Handles are like immutable pointers to those OS objects

- You cannot operate on them with arithmetic operations
- You can store it and use it later in the program to refer to the same object
- Sometimes you can check if valid against `INVALID_HANDLE_VALUE`

Example

- The `CreateWindowEx` function returns an `HWND`, a handle to the window
- To do anything to that window (such as `DestroyWindow`), use that handle

FILE-SYSTEM FUNCTIONS

`CreateFile`, `ReadFile`, `WriteFile`

- Normal file input/output

`CreateFileMapping`, `MapViewOfFile`

- Frequently used by malware, loads file contents into RAM
  - `CreateFileMapping` creates a file-mapping object
  - `MapViewOfFile` maps the view into the process address space (i.e., make the contents visible) and returns a pointer to the base address of file in memory for access
- Can be used to execute a file without using the Windows loader

SPECIAL FILES

Special paths

- `\\server\share\file.txt`
  - Programs can access a shared resource on another computer
  - These are UNC paths (Universal Naming Convention)
- `\\?\` prefix for extended paths (both local and UNC)
  - Bypasses normal Win32 path processing, allowing extended-length paths (>260 chars)

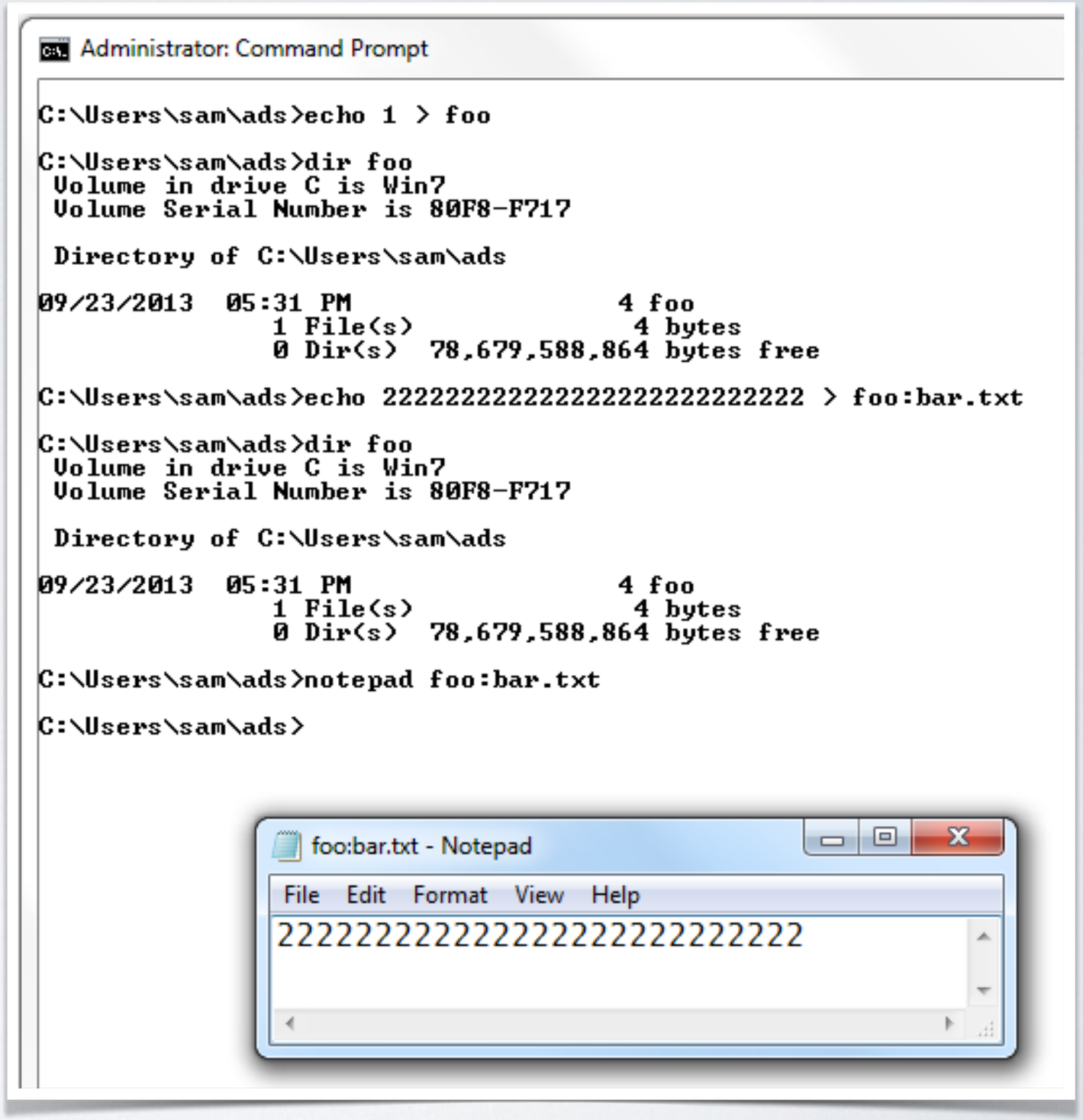
Namespaces

- Special folders in the Windows file system
- `\` Lowest namespace, contains everything
- `\\.\` Device namespace used for direct disk I/O
  - The Witty worm (2004) wrote to `\\.\PhysicalDisk1` to corrupt the disk

SPECIAL FILES

Alternate Data Streams

- Second stream of data attached to a filename
- `File.txt:otherfile.txt`
- The stream is kept in addition to the primary, visible content of the file.
- Feature of NTFS filesystem



WINDOWS REGISTRY

Store OS and program configuration settings

- Desktop background, mouse preferences, etc.

Malware may use the registry for persistence

- Have malware re-start when the system reboots

The registry comes with **5 root keys**:

Registry Root Keys

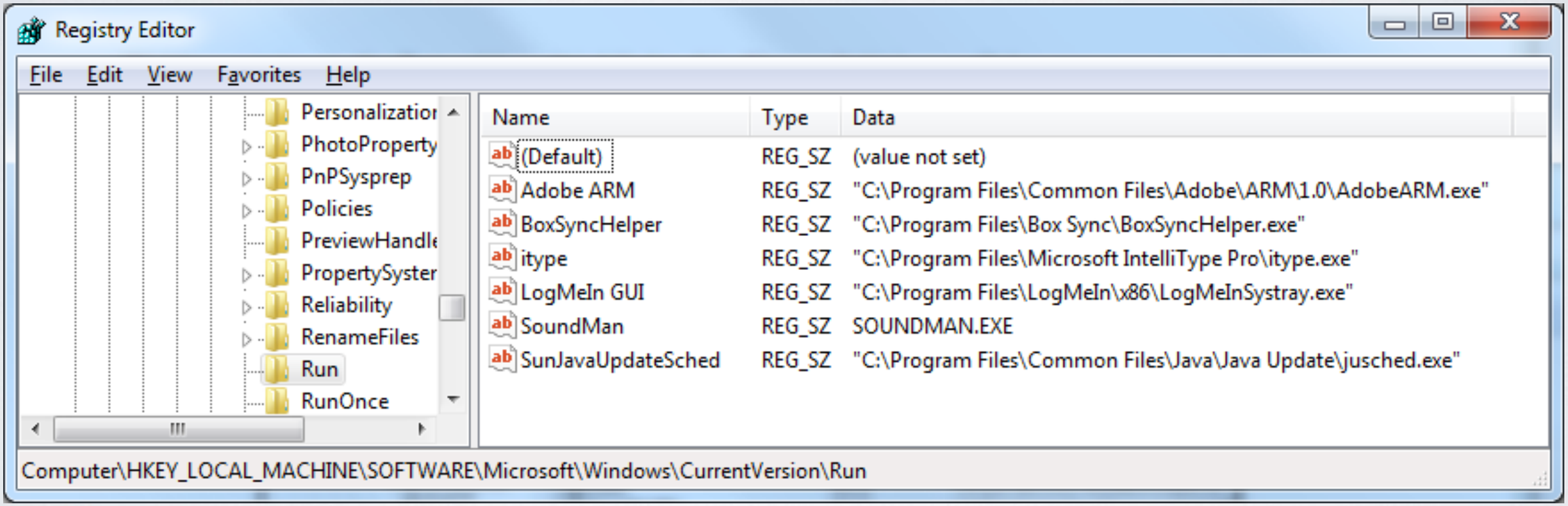
The registry is split into the following five root keys:

- **HKEY_LOCAL_MACHINE (HKLM)**. Stores settings that are global to the local machine
- **HKEY_CURRENT_USER (HKCU)**. Stores settings specific to the current user
- **HKEY_CLASSES_ROOT**. Stores information defining types
- **HKEY_CURRENT_CONFIG**. Stores settings about the current hardware configuration, specifically differences between the current and the standard configuration
- **HKEY_USERS**. Defines settings for the default user, new users, and current users

EXAMPLE

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`

- Executables that start when a user logs on



COMMON REGISTRY FUNCTIONS

`RegOpenKeyEx`

- Opens a registry key for editing and querying

`RegSetValueEx`

- Adds a new value to the registry & sets its data

`RegGetValue`

- Returns the data for a value entry in the Registry

Documentation often omits the trailing W (wide-char) or A (ASCII) character in a call like `RegOpenKeyExW`

EX, A, AND W SUFFIXES

- From Chapter 2 of the book

FUNCTION NAMING CONVENTIONS

When evaluating unfamiliar Windows functions, a few naming conventions are worth noting because they come up often and might confuse you if you don't recognize them. For example, you will often encounter function names with an `Ex` suffix, such as `CreateWindowEx`. When Microsoft updates a function and the new function is incompatible with the old one, Microsoft continues to support the old function. The new function is given the same name as the old function, with an added `Ex` suffix. Functions that have been significantly updated twice have two `Ex` suffixes in their names.

Many functions that take strings as parameters include an A or a W at the end of their names, such as `CreateDirectoryW`. This letter does *not* appear in the documentation for the function; it simply indicates that the function accepts a string parameter and that there are two different versions of the function: one for ASCII strings and one for wide character strings. Remember to drop the trailing `A` or `W` when searching for the function in the Microsoft documentation.

REGISTRY CODE

```
0040286F   push    2                   ; samDesired
00402871   push    eax                 ; ulOptions
00402872   push    offset SubKey       ; "Software\\Microsoft\\Windows\\CurrentVersion\\Run"
00402877   push    HKEY_LOCAL_MACHINE ; hKey
0040287C ❶call     esi ; RegOpenKeyExW
0040287E   test    eax, eax
00402880   jnz     short loc_4028C5
00402882
00402882 loc_402882:
00402882   lea     ecx, [esp+424h+Data]
00402886   push    ecx                 ; lpString
00402887   mov     bl, 1
00402889 ❷call     ds:lstrlenW
0040288F   lea     edx, [eax+eax+2]
00402893 ❸push     edx                 ; cbData
00402894   mov     edx, [esp+428h+hKey]
00402898 ❹lea      eax, [esp+428h+Data]
0040289C   push    eax                 ; lpData
0040289D   push    1                   ; dwType
0040289F   push    0                   ; Reserved
004028A1 ❺lea      ecx, [esp+434h+ValueName]
004028A8   push    ecx                 ; lpValueName
004028A9   push    edx                 ; hKey
004028AA   call    ds:RegSetValueExW
```

REGISTRY CODE

5 parameters for the call

Location for storing the output handle

Access level

```
lea  ecx, [esp+7E8h+phkResult]
push ecx                                ; phkResult
push 20006h                             ; samDesired KEY_WRITE
push 0                                  ; ulOptions
push offset aSoftwareMicros             ; Software\Microsoft\Windows\CurrentVersion\Run
push HKEY_CURRENT_USER                  ; hKey
call ds:RegOpenKeyExW
```

Syntax

C++ Copy

```
LSTATUS RegOpenKeyExA(
  HKEY   hKey,
  LPCSTR lpSubKey,
  DWORD  ulOptions,
  REGSAM samDesired,
  PHKEY  phkResult
);
```

REGISTRY CODE

```
mov   ecx, [esp+7E8h+phkResult]
sub   eax, edx
sar   eax, 1
lea   edx, ds:4[eax*4]
push  edx                       ; cbData
lea   eax, [esp+7ECh+pszPath]
push  eax                       ; lpData
push  REG_SZ                    ; dwType
push  0                         ; Reserved
push  offset ValueName          ; "System"
push  ecx                       ; hKey
call  ds:RegSetValueExW
 …
mov   edx, [esp+7E8h+phkResult]
push  edx                        ; hKey
call  ds:RegCloseKey
```

Size of data to be stored

Pointer to data to be stored

Type of data to be stored

Name of value

Handler to key

NETWORK API

Berkeley Compatible Sockets

Winsock libraries, primarily in `ws2_32.dll`

- Almost identical in Windows and Unix

Function	Description
`socket`	Creates a socket
`bind`	Attaches a socket to a particular port, prior to the `accept` call
`listen`	Indicates that a socket will be listening for incoming connections
`accept`	Opens a connection to a remote socket and accepts the connection
`connect`	Opens a connection to a remote socket; the remote socket must be waiting for the connection
`recv`	Receives data from the remote socket
`send`	Sends data to the remote socket

NOTE

The `WSAStartup` function must be called before any other networking functions in order to allocate resources for the networking libraries. When looking for the start of network connections while debugging code, it is useful to set a breakpoint on `WSAStartup`, because the start of networking should follow shortly.

SERVER AND CLIENT SIDES

Server side

- Maintains an open socket waiting for connections
- Calls, in order, `socket`, `bind`, `listen`, `accept`
- Then `send` and `recv` as necessary

Client side

- Connects to a waiting socket
- Calls, in order, `socket`, `connect`
- Then `send` and `recv` as necessary

THE WININET API

- Higher-level API than Winsock
- Functions in `Wininet.dll`
- Implements Application-layer protocols like HTTP and FTP
- `InternetOpen` – initializes use of api
- `InternetOpenURL` –connects to a URL
- `InternetReadFile` –reads data from a URL

SAPIENZA UNIVERSITÀ DI ROMA

EXECUTION MECHANISMS

While `jmp/call` transfer control within the current thread, Windows provides mechanisms to execute code in other threads or processes, and to coordinate such execution:

- DLLs
- Processes
- Threads
- Mutexes
- Services
- Component Object Model (COM)
- Exceptions

DYNAMIC LINK LIBRARIES (DLLS)

- DLLs enable sharing code to be used by other applications
- Using DLLs included in Windows makes code smaller. Then, software companies can also make their custom DLLs
- Malware authors use DLLs in multiple ways
  - Use Windows DLLs for interacting with OS
  - Store malicious code in DLL (e.g., load the DLL into another process)
  - Use third-party DLLs (e.g., use Firefox DLL to connect to a server, instead of Windows API)

BASIC DLL STRUCTURE

- DLLs are very similar to EXEs and share the PE file format
- A single flag in PE header discriminates between DLL and EXE
- DLLs have more exports and fewer imports
- `DllMain` a highly relevant function for initialization and cleanup
  - Not exported, but specified as the entry point in the PE Header
  - Called when a function loads or unloads the library, and on thread creation and termination

SAPIENZA UNIVERSITÀ DI ROMA

PROCESSES

- As we said, a program being executed by Windows is a process and each process has its own resources (handles, memory, etc.)
- Each process has one or more execution units called **threads**
- Malware can run in a standalone process or also have some code execute in another process

CREATING A NEW PROCESS

`CreateProcess`

- `CommandLine` contains the command line to execute for the desired program
- `STARTUPINFO` parameter contains handles for standard input, standard output, and standard error streams. Could be set to a socket for a remote shell!

Syntax

C++ Copy

```
BOOL CreateProcessA(
  LPCSTR                lpApplicationName,
  LPSTR                 lpCommandLine,
  LPSECURITY_ATTRIBUTES lpProcessAttributes,
  LPSECURITY_ATTRIBUTES lpThreadAttributes,
  BOOL                  bInheritHandles,
  DWORD                 dwCreationFlags,
  LPVOID                lpEnvironment,
  LPCSTR                lpCurrentDirectory,
  LPSTARTUPINFOA        lpStartupInfo,
  LPPROCESS_INFORMATION lpProcessInformation
);
```

THREADS

Processes are containers

- Each process contains one or more threads

Threads are what Windows actually executes

- Independent sequences of instructions
  - Independently scheduled by the OS; may execute concurrently or in parallel
- Threads within a process share the same memory space
- Each thread has its own stack and CPU execution **context**
  - Context: register state, including instruction and stack pointers
  - The OS saves and restores each thread's CPU context in a dedicated structure when switching between threads on a CPU core
- Created with `CreateThread` to run from a designated start function

EXCEPTIONS

- Exceptions are caused by hardware faults (such as division by zero or invalid memory access) or raised explicitly by software
- Windows provides **Structured Exception Handling** (SEH) to dispatch exceptions. When an exception occurs, execution may transfer to an appropriate *Structured Exception Handler*
  - *FS is a segment register used to access thread-specific data*

Example 8-13. Storing exception-handling information in fs:0

```
01006170  push  1offset loc_10061C0
01006175  mov     eax, large fs:0
0100617B  push  2eax
0100617C  mov     large fs:0, esp
```

- When an exception occurs, Windows uses `FS:[0]` to find the first SEH registration record on the stack, then traverses the chain to locate a suitable handler

MUTEXES

- Mutexes are synchronization objects to coordinate access to shared resources
- They are inter-process communication (IPC) objects, hence they can be shared across threads and processes for coordination
- Mutexes can be named, and such names are often hard-coded
  - Good source of IoCs to identify malware
  - Malware may use them to check whether another instance is already running

FUNCTIONS FOR MUTEXES

`CreateMutex`

- Creates a mutex (or opens an existing named mutex)

`OpenMutex`

- Opens an existing named mutex

`WaitForSingleObject`

- Waits to acquire ownership of a mutex (blocking call)
- Any subsequent threads attempting to gain access to it must wait

`ReleaseMutex`

- Called when a thread is done using the mutex

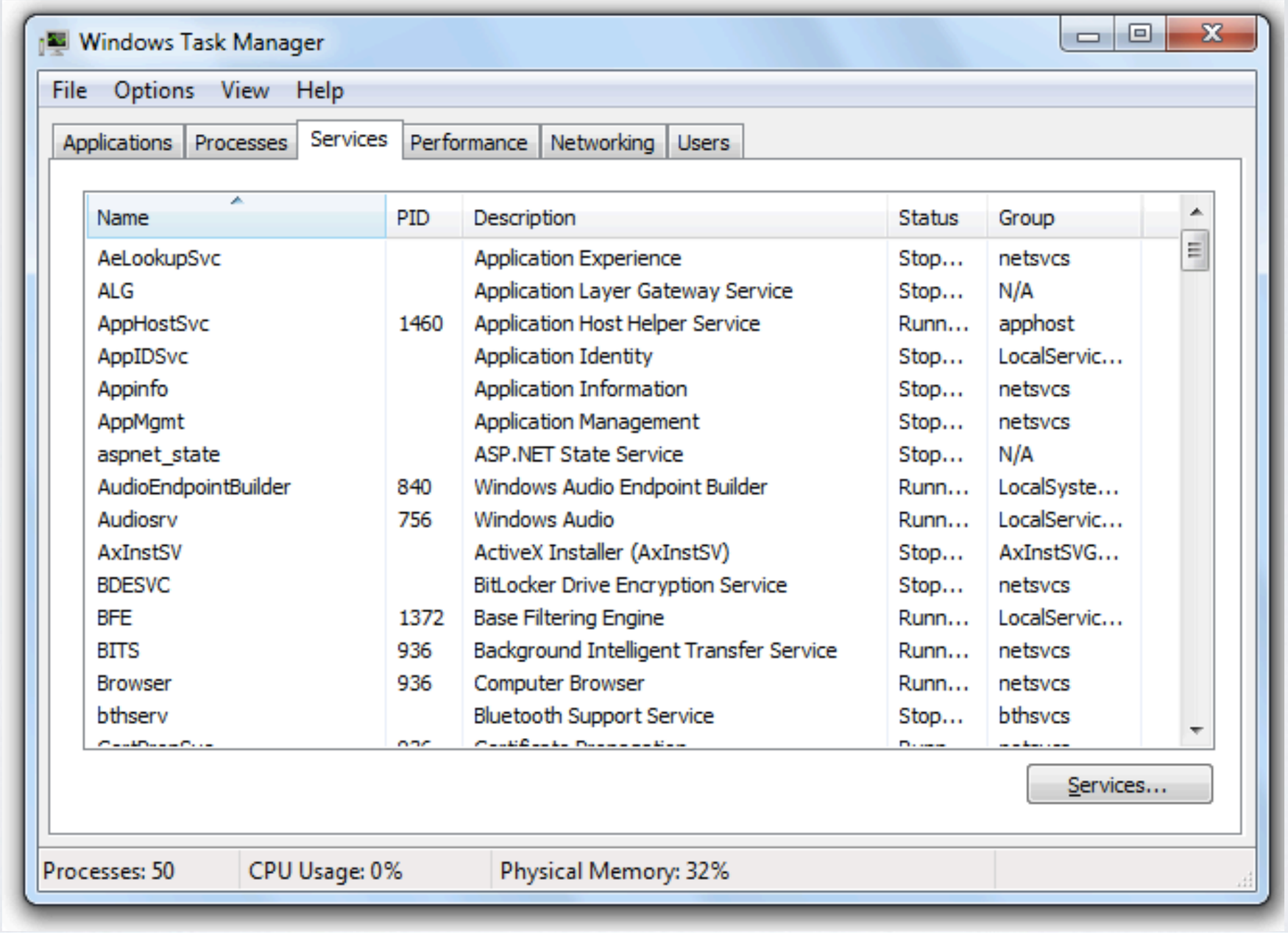
MALWARE SINGLE-INSTANCE CHECK

- `OpenMutex` checks if mutex “HGL345” exists
- If not, it is created with `CreateMutex`
- `test eax, eax`
  sets `z` flag if `eax` is zero

```
00401007   push  1F0001h          ; dwDesiredAccess
0040100C  1call  ds:__imp__OpenMutexW@12 ;
OpenMutexW(x,x,x)
00401012  2test  eax, eax
00401014  3jz    short loc_40101E
00401016   push  0                ; int
00401018  4call  ds:__imp__exit
0040101E   push  offset Name      ; "HGL345"
00401023   push  0                ; bInitialOwner
00401025   push  0                ; lpMutexAttributes
00401027  5call  ds:__imp__CreateMutexW@12 ;
CreateMutexW(x,x,x)
```

SERVICES

- Services are background processes running without user input



SERVICES

- These background processes are managed by the Windows Service Control Manager (SCM)
  - They can run independently of user logon and start automatically when Windows boots
- Each service runs under a configured security account
  - Some run as LocalSystem (`SYSTEM`) with extensive local privileges, including access to many protected system resources
  - Others run under restricted service accounts or specific user accounts
  - Service creation and configuration require appropriate privileges
- Malware can install services to establish persistence, potentially with elevated privileges

SERVICE API FUNCTIONS

`OpenSCManager`

- Returns a handle to the Service Control Manager

`CreateService`

- Adds a new service to the Service Control Manager
- Can specify whether the service will start automatically at boot time

`StartService`

- Starts a stopped service (e.g., one set to start manually)

SVCHOST.EXE

The `dwServiceType` for `CreateService` determines the `SERVICE_` type:

- `WIN32_OWN_PROCESS (0x10)`: dedicated process
- `WIN32_SHARE_PROCESS (0x20)`: can share a process with other services

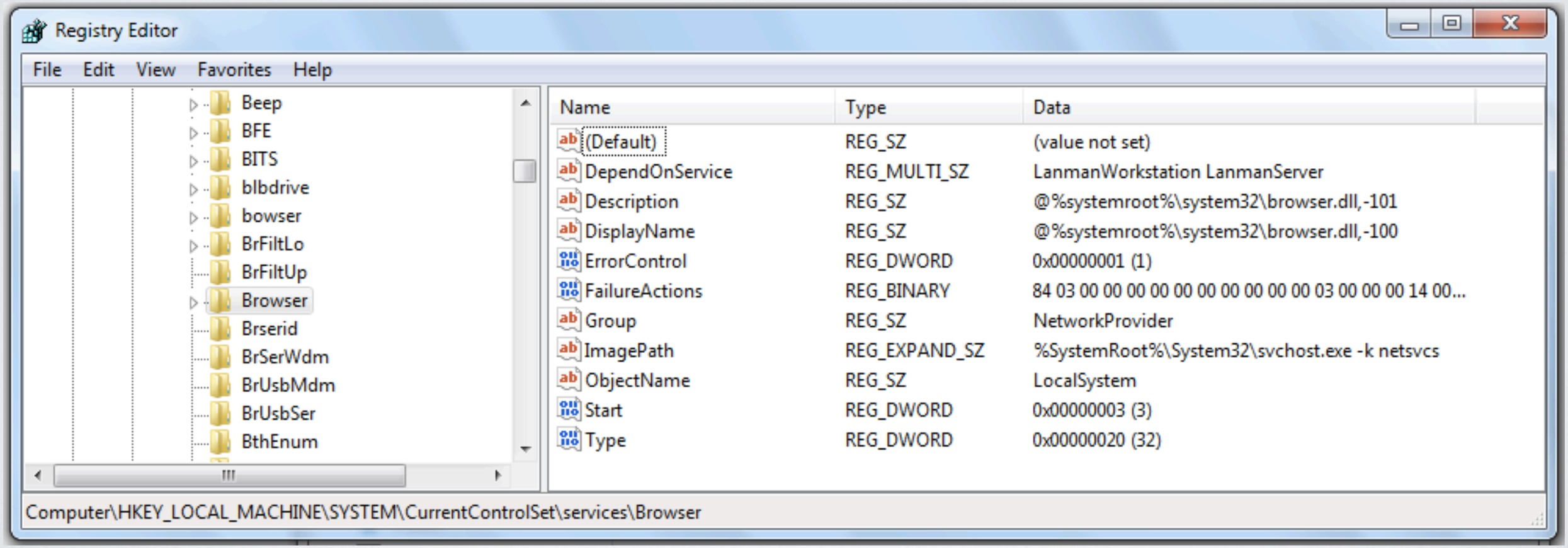
Notably, **svchost.exe** is a Windows service-host executable that loads and runs services implemented as DLLs

- Different services may run within the same svchost.exe instance or in separate instances. The actual service code resides in a DLL, not in svchost.exe itself
- Malware analysts shall inspect the service configuration and its `ServiceDll` entry to identify the DLL being executed
- While svchost.exe is intended for Windows services, that does not prevent malware from disguising malicious activity through it

SERVICE INFORMATION IN THE REGISTRY

`HKLM\System\CurrentControlSet\Services\<ServiceName>`

- Start value = 0x03 for "Load on Demand"
- Type = 0x20 for WIN32_SHARE_PROCESS
- ImagePath = executable/driver path



COMPONENT OBJECT MODEL (COM)

- Allows software components to expose functionality through standardized interfaces, including across process boundaries
- Every thread using COM generally initializes the COM library by calling `OleInitialize` or `CoInitializeEx`
- COM classes and interfaces are identified and accessed via Globally Unique Identifiers (GUIDs). Main types include:
  - Class Identifiers (CLSIDs): identify classes of objects
    - in Registry under HKEY_CLASSES_ROOT\CLSID
  - Interface Identifiers (IIDs): identify interfaces exposed by objects
    - in Registry under HKEY_CLASSES_ROOT\Interface

COM OBJECT ACTIVATION

- Applications can instantiate COM objects using CoCreateInstance, specifying a CLSID and a requested IID.
- Windows typically uses the CLSID registration to locate the corresponding implementation:
  - `InprocServer32` → DLL loaded into the calling process
  - `LocalServer32` → EXE running in a separate process
- Malware relevance
  - Malware can use COM objects to access Windows functionality
  - **COM hijacking**: malicious modifications to COM registrations can cause legitimate applications to load attacker-controlled code

ASSEMBLY PATTERNS

The following slides cover some assembly constructs that may be relevant during a quick code inspection. They mean to complement what seen in the hands-on lab sessions on x86 assembly.

GLOBAL VS. LOCAL VARIABLES

- Global variables can be accessed and used by any function in a program
- Local variables can be accessed only by the function in which they are defined
- Both global and local variables are declared similarly in C, but they look completely different in assembly

GLOBAL VS. LOCAL VARIABLES

```
int x = 1;
int y = 2;

void main()
{
    x = x+y;
    printf("Total = %d\n", x);
}
```

```
00401003        mov     eax, dword_40CF60
00401008        add     eax, dword_40C000
0040100E        mov     dword_40CF60, eax ❶
00401013        mov     ecx, dword_40CF60
00401019        push    ecx
0040101A        push    offset aTotalD  ;"total = %d\n"
0040101F        call    printf
```

GLOBAL VS. LOCAL VARIABLES

```
void main()
{
   int x = 1;
   int y = 2;

   x = x+y;
   printf("Total = %d\n", x);
}
```

```
00401006        mov     dword ptr [ebp-4], 1
0040100D        mov     dword ptr [ebp-8], 2
00401014        mov     eax, [ebp-4]
00401017        add     eax, [ebp-8]
0040101A        mov     [ebp-4], eax
0040101D        mov     ecx, [ebp-4]
00401020        push    ecx
00401021        push    offset aTotalD  ; "total = %d\n"
00401026        call    printf
```

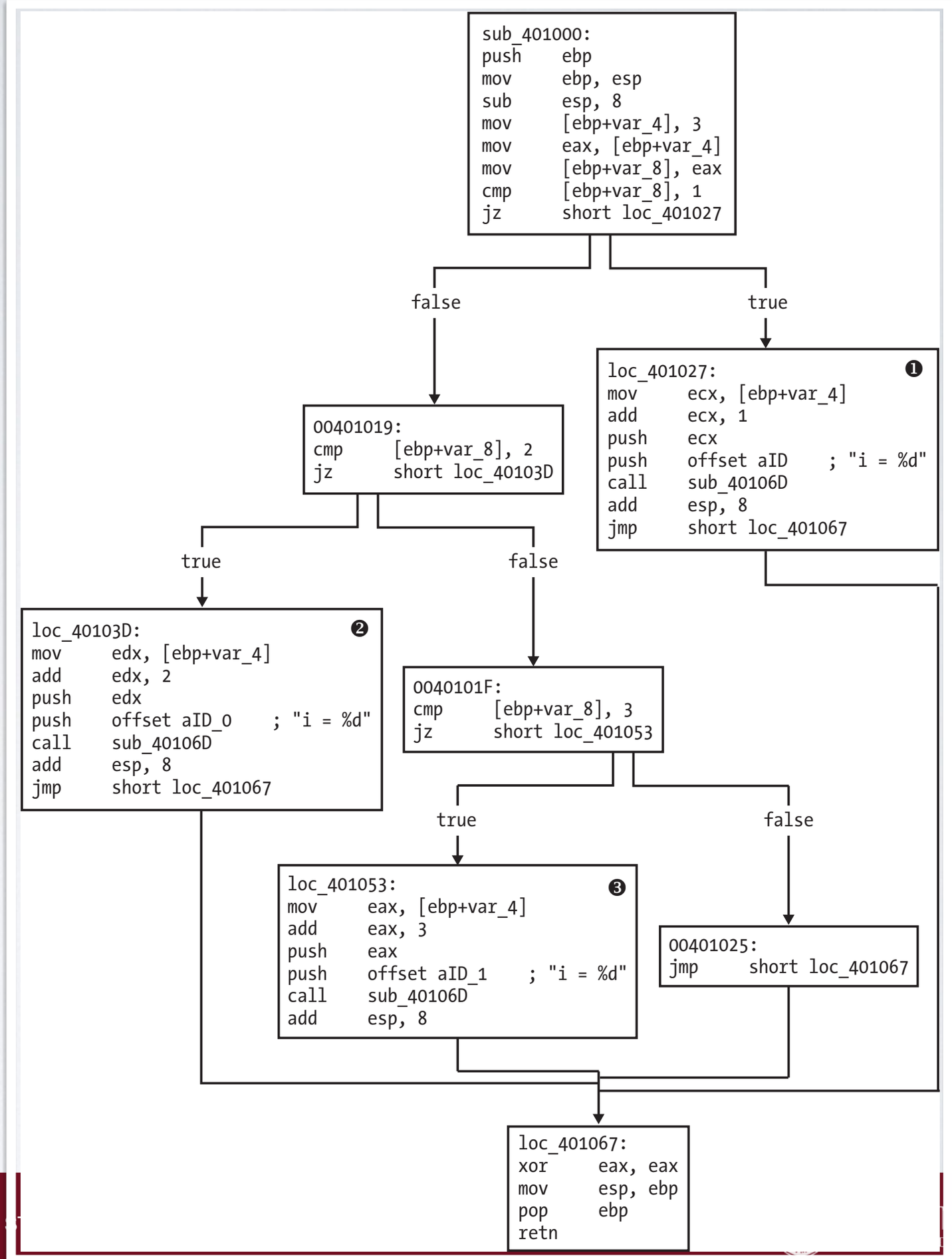
SWITCH

```
switch(i)
{
   case 1:
      printf("i = %d", i+1);
      break;
   case 2:
      printf("i = %d", i+2);
      break;
   case 3:
      printf("i = %d", i+3);
      break;
   default:
      break;
}
```

```
00401013        cmp     [ebp+var_8], 1
00401017        jz      short loc_401027 ❶
00401019        cmp     [ebp+var_8], 2
0040101D        jz      short loc_40103D
0040101F        cmp     [ebp+var_8], 3
00401023        jz      short loc_401053
00401025        jmp     short loc_401067 ❷
00401027 loc_401027:
00401027        mov     ecx, [ebp+var_4] ❸
0040102A        add     ecx, 1
0040102D        push    ecx
0040102E        push    offset unk_40C000 ; i = %d
00401033        call    printf
00401038        add     esp, 8
0040103B        jmp     short loc_401067
0040103D loc_40103D:
0040103D        mov     edx, [ebp+var_4] ❹
00401040        add     edx, 2
00401043        push    edx
00401044        push    offset unk_40C004 ; i = %d
00401049        call    printf
0040104E        add     esp, 8
00401051        jmp     short loc_401067
00401053 loc_401053:
00401053        mov     eax, [ebp+var_4] ❺
00401056        add     eax, 3
00401059        push    eax
0040105A        push    offset unk_40C008 ; i = %d
0040105F        call    printf
00401064        add     esp, 8
```

SWITCH

```
switch(i)
{
    case 1:
        printf("i = %d", i+1);
        break;
    case 2:
        printf("i = %d", i+2);
        break;
    case 3:
        printf("i = %d", i+3);
        break;
    default:
        break;
}
```



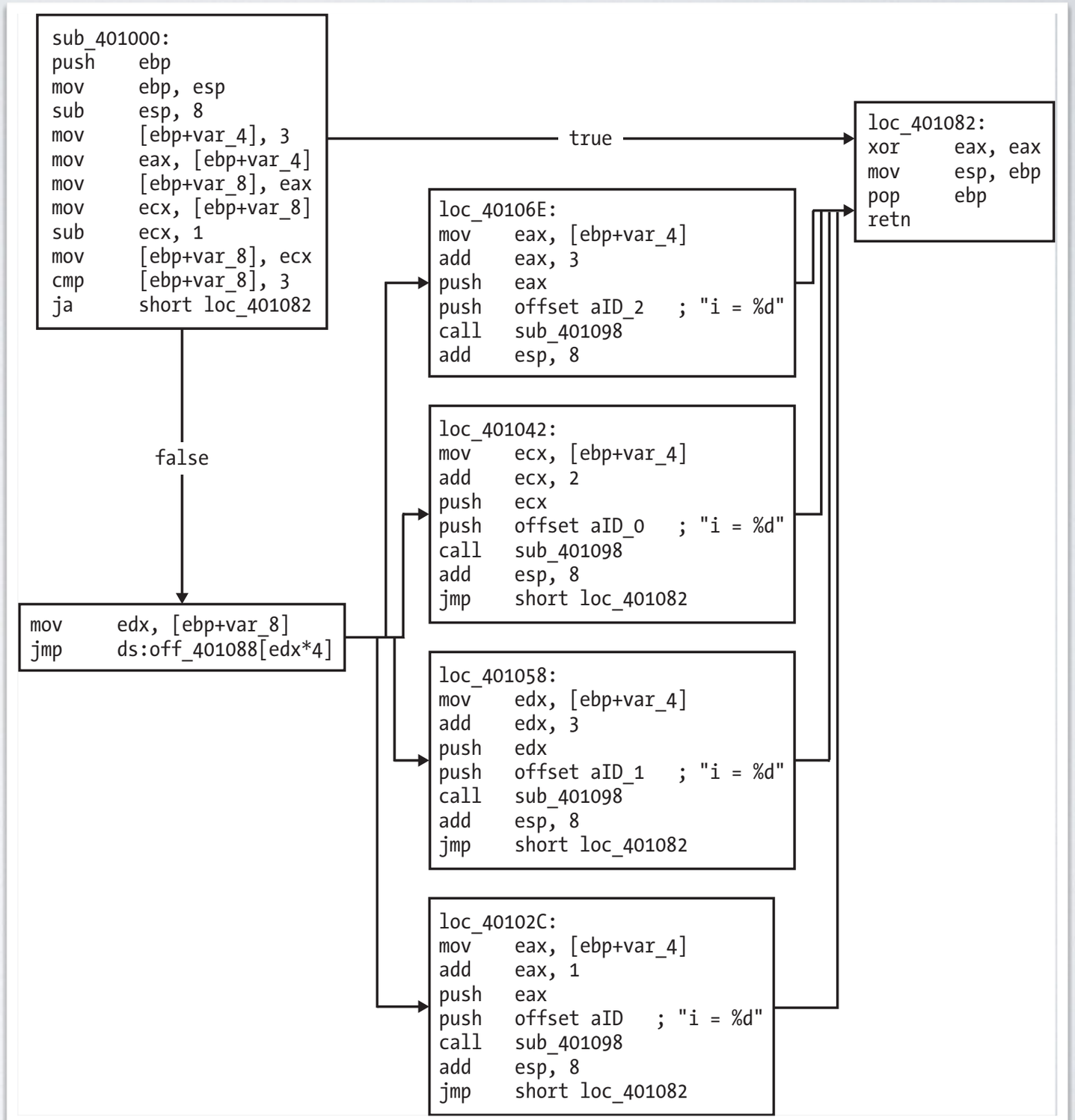
SWITCH

```
switch(i)
{
   case 1:
      printf("i = %d", i+1);
      break;
   case 2:
      printf("i = %d", i+2);
      break;
   case 3:
      printf("i = %d", i+3);
      break;
   case 4:
      printf("i = %d", i+3);
      break;
   default:
      break;
}
```

```
00401016          sub     ecx, 1
00401019          mov     [ebp+var_8], ecx
0040101C          cmp     [ebp+var_8], 3
00401020          ja      short loc_401082
00401022          mov     edx, [ebp+var_8]
00401025          jmp     ds:off_401088[edx*4] ❶
0040102C    loc_40102C:
             ...
00401040          jmp     short loc_401082
00401042    loc_401042:
             ...
00401056          jmp     short loc_401082
00401058    loc_401058:
             ...
0040106C          jmp     short loc_401082
0040106E    loc_40106E:
             ...
00401082    loc_401082:
00401082          xor     eax, eax
00401084          mov     esp, ebp
00401086          pop     ebp
00401087          retn
00401087    _main   endp
00401088 ❷off_401088  dd offset loc_40102C
0040108C              dd offset loc_401042
00401090              dd offset loc_401058
00401094              dd offset loc_40106E
```

SWITCH

```
switch(i)
{
   case 1:
      printf("i = %d", i+1);
      break;
   case 2:
      printf("i = %d", i+2);
      break;
   case 3:
      printf("i = %d", i+3);
      break;
   case 4:
      printf("i = %d", i+3);
      break;
   default:
      break;
}
```



ARRAYS

```
int b[5] = {123,87,487,7,978};
void main()
{
   int i;
   int a[5];

   for(i = 0; i<5; i++)
   {
      a[i] = i;
      b[i] = i;
   }
}
```

```
00401006          mov     [ebp+var_18], 0
0040100D          jmp     short loc_401018
0040100F loc_40100F:
0040100F          mov     eax, [ebp+var_18]
00401012          add     eax, 1
00401015          mov     [ebp+var_18], eax
00401018 loc_401018:
00401018          cmp     [ebp+var_18], 5
0040101C          jge     short loc_401037
0040101E          mov     ecx, [ebp+var_18]
00401021          mov     edx, [ebp+var_18]
00401024          mov     [ebp+ecx*4+var_14], edx ❶
00401028          mov     eax, [ebp+var_18]
0040102B          mov     ecx, [ebp+var_18]
0040102E          mov     dword_40A000[ecx*4], eax ❷
00401035          jmp     short loc_40100F
```

STRUCTS

```
struct my_structure { ❶
     int x[5];
     char y;
     double z;
};

struct my_structure *gms; ❷

void test(struct my_structure *q)
{
     int i;
     q->y = 'a';
     q->z = 15.6;
     for(i = 0; i<5; i++){
          q->x[i] = i;
     }
}

void main()
{
     gms = (struct my_structure *) malloc(
     sizeof(struct my_structure));
     test(gms);
}
```

```
00401050        push    ebp
00401051        mov     ebp, esp
00401053        push    20h
00401055        call    malloc
0040105A        add     esp, 4
0040105D        mov     dword_40EA30, eax
00401062        mov     eax, dword_40EA30
00401067        push    eax ❶
00401068        call    sub_401000
0040106D        add     esp, 4
00401070        xor     eax, eax
00401072        pop     ebp
00401073        retn
```

STRUCTS

```
struct my_structure { ❶
     int x[5];
     char y;
     double z;
};

struct my_structure *gms; ❷

void test(struct my_structure *q)
{
     int i;
     q->y = 'a';
     q->z = 15.6;
     for(i = 0; i<5; i++){
          q->x[i] = i;
     }
}

void main()
{
     gms = (struct my_structure *) malloc(
     sizeof(struct my_structure));
     test(gms);
}
```

```
00401000          push    ebp
00401001          mov     ebp, esp
00401003          push    ecx
00401004          mov     eax,[ebp+arg_0]
00401007          mov     byte ptr [eax+14h], 61h
0040100B          mov     ecx, [ebp+arg_0]
0040100E          fld     ds:dbl_40B120 ❶
00401014          fstp    qword ptr [ecx+18h]
00401017          mov     [ebp+var_4], 0
0040101E          jmp     short loc_401029
00401020 loc_401020:
00401020          mov     edx,[ebp+var_4]
00401023          add     edx, 1
00401026          mov     [ebp+var_4], edx
00401029 loc_401029:
00401029          cmp     [ebp+var_4], 5
0040102D          jge     short loc_40103D
0040102F          mov     eax,[ebp+var_4]
00401032          mov     ecx,[ebp+arg_0]
00401035          mov     edx,[ebp+var_4]
00401038          mov     [ecx+eax*4],edx ❷
0040103B          jmp     short loc_401020
0040103D loc_40103D:
0040103D          mov     esp, ebp
0040103F          pop     ebp
00401040          retn
```